

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	LOKESH S	Reg. No.:	192572151
Section / Batch:	CSE(AI) / 2ND YEAR	Date:	10/08/2026

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Analyze a processor datapath that performs register transfers and logical operations such as XOR, NOT, and comparison, and examine the control signals and execution sequence required for each operation.
2. Develop a bus-based system for transferring data between the processor, memory, and I/O devices, and analyze how bus width, transfer rate, and bus contention influence system performance.
3. Simulate a pipelined processor with arithmetic, memory, and branch instructions, and evaluate the effects of pipeline depth on execution time, throughput, and overall speedup.
4. Construct a pipeline containing instruction dependencies and branch conditions, and analyze the performance impact of data and control hazards using stalling, forwarding, and branch prediction techniques.
5. Develop a superscalar processor model that supports multiple execution units and out-of-order instruction processing, and evaluate how instruction scheduling and resource allocation improve instruction throughput.

Code of Conduct

*I **LOKESH S (192572151)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: LOKESH S

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator Faculty Incharge

1. Processor Datapath for Register Transfers, XOR, NOT, and Comparison

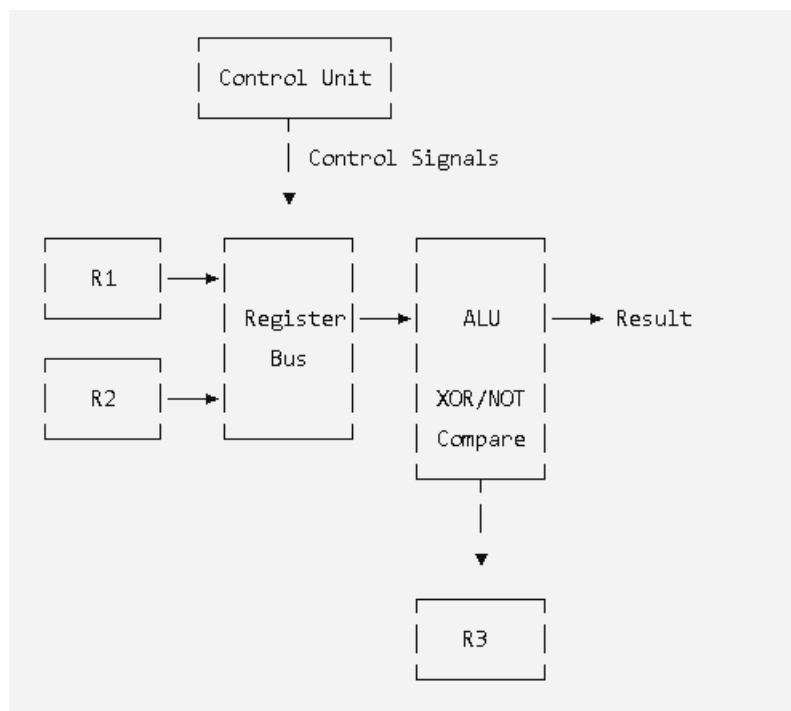
Introduction

A processor Datapath is the hardware path through which data moves and gets processed inside a CPU. It consists mainly of registers, ALU, multiplexers, buses, and control units.

Consider a processor with:

- Registers: R1, R2, R3
- ALU supporting XOR, NOT, and comparison
- Program Counter (PC)
- Instruction Register (IR)
- Register file
- Control Unit
- Internal data bus

A simplified Datapath is:



A. Register Transfer Operation

Suppose the instruction is:

$R3 \leftarrow R1$

The processor must transfer the contents of R1 into R3.

Execution sequence

1. Select R1 as the source register.
2. Enable R1 output.
3. Place its value on the internal bus.
4. Enable R3 as the destination.
5. Load the bus value into R3.

Conceptually:

$R1 \rightarrow \text{Internal Bus} \rightarrow R3$

Control signals

Control Signal	Function
R1out	Places R1 value on bus
Bus Enable	Enables internal bus
R3in	Loads bus value into R3

B. XOR Operation

Consider:

$R3 \leftarrow R1 \text{ XOR } R2$

The ALU receives two operands:

Operand A = R1

Operand B = R2

The ALU performs:

$R3 = R1 \text{ XOR } R2$

For example:

$R1 = 1010$

$R2 = 1100$

$R1 \text{ XOR } R2$

$= 0110$

Execution sequence

1. Read R1.
2. Read R2.
3. Send both operands to the ALU.
4. Control unit selects XOR operation.
5. ALU generates the result.
6. Store the result in R3.

Control signals

Signal	Function
R1out	Selects R1
R2out	Selects R2
ALU_XOR	Selects XOR operation

Signal	Function
R3in	Stores result in R3
Clock	Synchronizes transfer

C. NOT Operation

Consider:

$R2 \leftarrow \text{NOT } R1$

If:

$R1 = 1010$

then:

$\text{NOT } R1 = 0101$

Execution

$R1 \rightarrow \text{ALU} \rightarrow \text{NOT} \rightarrow R2$

The control unit activates:

R1out

ALU_NOT

R2in

D. Comparison Operation

Comparison determines whether one value is equal to, greater than, or less than another.

For:

CMP R1, R2

the ALU effectively calculates:

$R1 - R2$

and examines status flags such as:

- Zero Flag (Z)
- Carry Flag (C)
- Sign/Negative Flag (N)
- Overflow Flag (V)

For example:

$R1 = 10$

$R2 = 10$

Analysis

The Datapath performs the actual data processing, while the control unit determines when and how each operation occurs. Efficient control-signal generation is essential because incorrect sequencing can cause incorrect register transfers or ALU results.

Conclusion

A processor Datapath provides the physical path for transferring and processing data. Register transfers use buses and register-enable signals, while logical operations use the ALU. Comparison operations additionally update status flags that can be used by conditional branch instructions.

2. Bus-Based System for Processor, Memory, and I/O

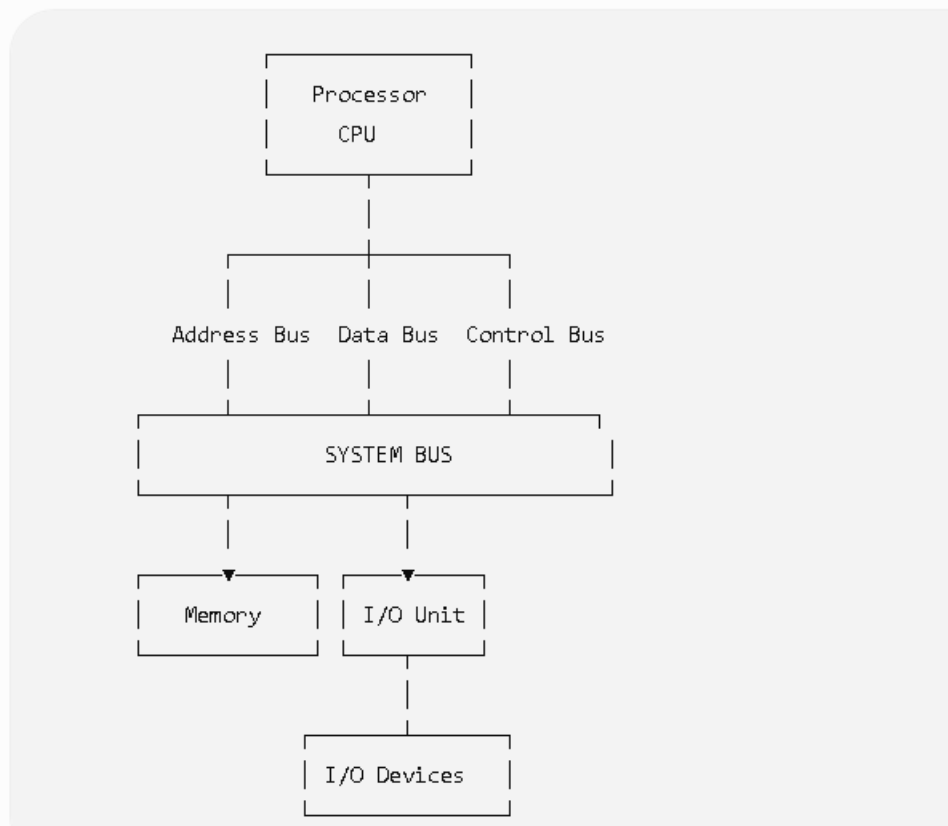
Introduction

A bus-based system provides a common communication path between the processor, memory, and I/O devices.

A typical system contains:

1. Data bus
2. Address bus
3. Control bus

System Architecture



A. Address Bus

The address bus identifies the memory location or I/O device involved in a transfer.

For an n-bit address bus:

Maximum addressable locations = 2^n

For example, a 32-bit address bus can theoretically address:

$$2^{32} = 4,294,967,296$$

address locations.

B. Data Bus

The data bus transfers the actual data.

A wider data bus allows more data to be transferred in one bus operation.

For example:

Data Bus	Data transferred per cycle
8-bit	1 byte
16-bit	2 bytes
32-bit	4 bytes
64-bit	8 bytes

If a 64-bit bus operates at 100 MHz and transfers one 64-bit word per cycle:

$$\begin{aligned}\text{Bandwidth} &= 64 \text{ bits} \times 100 \text{ million cycles/sec} \\ &= 6.4 \text{ Gbit/s} \\ &= 800 \text{ MB/s}\end{aligned}$$

Actual throughput may be lower because of protocol overhead, wait states, arbitration, and other limitations.

C. Control Bus

The control bus carries signals such as:

- Read
- Write
- Memory Enable
- I/O Enable
- Interrupt
- Bus Request
- Bus Grant
- Clock
- Reset

Bus Width and Performance

Bus width directly influences the amount of information transferred per cycle.

A wider bus generally increases bandwidth:

$$\text{Bandwidth} \propto \text{Bus Width} \times \text{Transfer Frequency}$$

For example:

$$32\text{-bit bus} \times 100 \text{ MHz}$$

has a theoretical raw transfer rate of:

$$3.2 \text{ Gbit/s}$$

while:

$$64\text{-bit bus} \times 100 \text{ MHz}$$

provides:

$$6.4 \text{ Gbit/s}$$

under the same one-transfer-per-cycle assumption.

Transfer Rate

Transfer rate determines how quickly data moves through the bus.

Increasing:

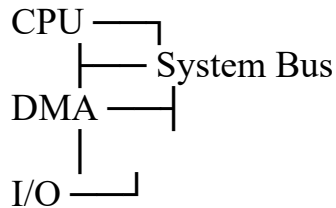
- bus frequency
- number of transfers per cycle

- bus width
can increase theoretical bandwidth.
However, simply increasing clock frequency does not guarantee proportional performance improvement because memory and I/O devices may be slower.

Bus Contention

Bus contention occurs when multiple devices attempt to use a shared bus simultaneously.

For example:



If both CPU and DMA request the bus simultaneously, an arbitration mechanism must determine who gets access.

Solutions

- Bus arbitration
- Priority-based access
- Round-robin scheduling
- Separate buses
- DMA controllers
- Bus segmentation

Performance Impact

Bus contention causes:

- Waiting time
- Reduced effective bandwidth
- Increased latency
- CPU stalls
- Lower system throughput

Conclusion

A bus-based system provides a simple communication structure between CPU, memory, and I/O devices. Increasing bus width and transfer rate improves theoretical bandwidth, but contention and slower components can limit actual performance. Therefore, efficient bus arbitration and system-level design are necessary.

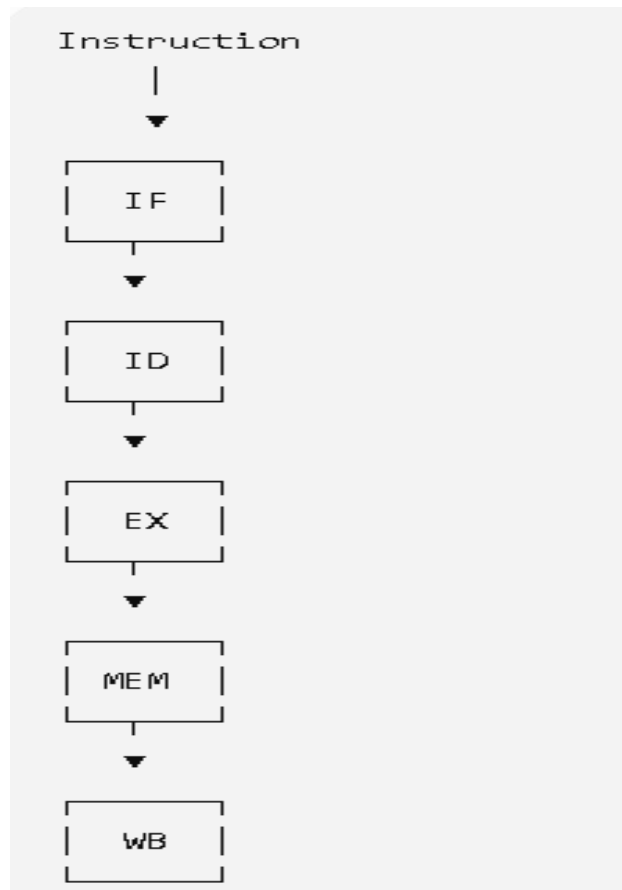
3. Pipelined Processor with Arithmetic, Memory, and Branch Instructions

Introduction

Pipelining divides instruction execution into multiple stages so that several instructions can be processed simultaneously.

A common five-stage pipeline is:

Stage	Function
IF	Instruction Fetch
ID	Instruction Decode
EX	Execute
MEM	Memory Access
WB	Write Back



Suppose the instructions are:

```
I1: ADD R1,R2,R3
I2: LOAD R4,0(R5)
I3: SUB R6,R7,R8
I4: BEQ R1,R0,LABEL
```

Pipeline Execution

Cycle	I1	I2	I3	I4
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	EX	ID	IF
5	WB	MEM	EX	ID
6		WB	MEM	EX
7			WB	MEM

This demonstrates overlapping execution.

Pipeline Depth

Non-pipelined execution = $5 \times T$

For n instructions:

Non-pipelined time $\approx 5nT$

For an ideal five-stage pipeline:

Pipeline time $\approx (n + 4)T$

For 100 instructions:

Non-pipelined = $500T$

Pipelined = $104T$

Ideal speedup:

Speedup = $500 / 104$

≈ 4.81

The speedup approaches the number of pipeline stages for a large number of instructions.

Effect of Pipeline Depth

Increasing pipeline depth can reduce the amount of work per stage, potentially allowing a higher clock frequency.

However, deeper pipelines also introduce:

- More pipeline registers
- Greater control complexity
- Higher branch misprediction penalty
- More hazard-management overhead
- Additional clocking overhead

Therefore, deeper pipelines do not automatically guarantee better overall performance.

Throughput

Pipeline throughput improves because multiple instructions are processed simultaneously.

In an ideal pipeline:

One instruction can complete every cycle after the pipeline becomes full.

Conclusion

Pipelining improves processor throughput by overlapping instruction execution.

However, pipeline depth must be carefully selected because deeper pipelines can increase clock frequency while also increasing hazard and branch penalties.

4. Pipeline Dependencies, Branches, and Hazards

Introduction

Pipeline hazards occur when the next instruction cannot safely execute in its normal pipeline position.

Three major types are:

1. Structural hazards
2. Data hazards
3. Control hazards

This question focuses mainly on data and control hazards.

A. Data Dependency

```
I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
```

Without Forwarding

```
Cycle:  1  2  3  4  5  6
I1:      IF ID EX MEM WB
I2:      IF ID ST ST EX ...
```

B. Forwarding

I1 ALU Result —————▶ I2 ALU Input
This reduces unnecessary stalls.

Benefit

Forwarding:

- Reduces RAW hazard penalties
- Improves throughput
- Reduces pipeline stalls

However, forwarding cannot eliminate every hazard.

C. Load-Use Hazard

```
I1: LOAD R1,0(R2)
I2: ADD R3,R1,R4
```

Therefore, even with forwarding, a stall may be required:

```
I1: IF ID EX MEM WB
I2:   IF ID ST EX MEM WB
```

Control Hazards

Control hazards occur because the processor does not immediately know which instruction should be fetched after a branch.

Example:

```
I1: BEQ R1,R2,LABEL
I2: ADD R3,R4,R5
I3: SUB R6,R7,R8
```

Branch Prediction

A branch predictor predicts whether a branch will be:

Taken

or:

Not Taken

Correct Prediction

If prediction is correct:

No major pipeline flush

and execution continues efficiently.

Incorrect Prediction

If prediction is wrong:

Incorrect instructions → Flush

→ Correct instructions fetched

This creates a branch misprediction penalty.

Comparison of Techniques

Technique	Main Purpose	Benefit	Limitation	
Stalling	Wait for required data	Simple	Reduces performance	
Forwarding	Send result directly	Reduces data stalls	Cannot solve every dependency	
Branch Prediction	Predict branch outcome	Reduces control stalls	Mispredictions cause flushes	

Performance Example

Suppose:

100 instructions

10% are branches

20% of branches are mispredicted

Penalty = 3 cycles

Mispredictions:

$$100 \times 10\% \times 20\% = 2$$

Penalty:

$$2 \times 3 = 6 \text{ cycles}$$

Conclusion

Data and control hazards can significantly reduce pipeline efficiency. Forwarding minimizes many data dependencies, while branch prediction reduces the cost of control hazards. Stalling remains necessary when the required data or branch information cannot be made available in time.

5. Superscalar Processor with Multiple Execution Units and Out-of-Order Processing

Introduction

A superscalar processor can issue and execute multiple instructions during a single clock cycle.

Unlike a scalar processor:

Scalar:

1 instruction/cycle

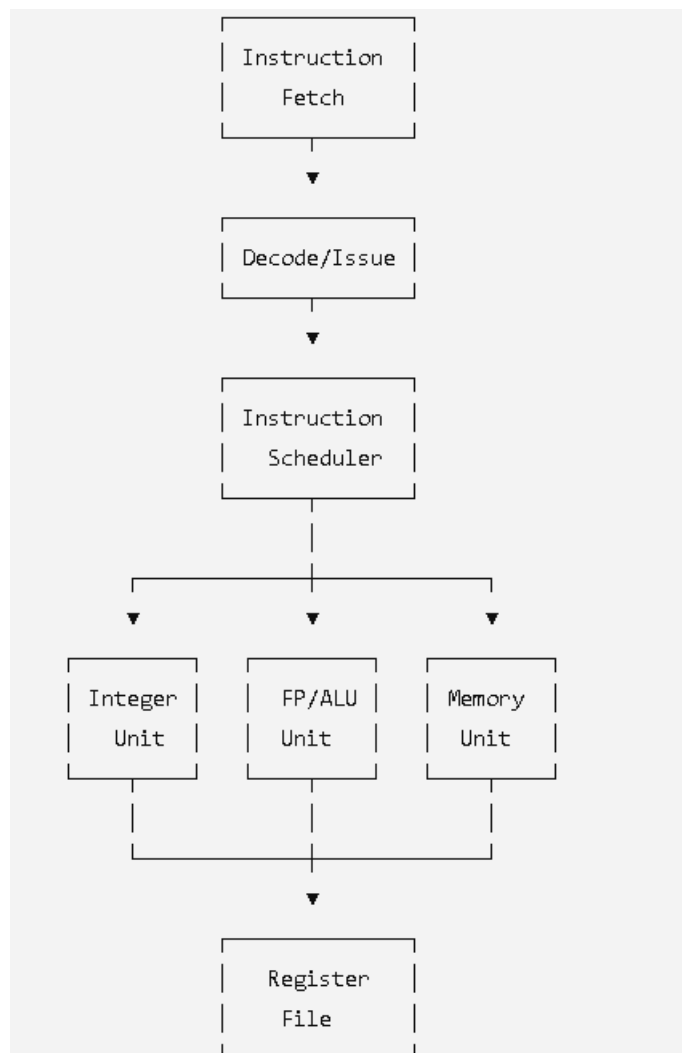
a superscalar processor may support:

2, 4, or more instructions/cycle

depending on available resources and instruction-level parallelism.

Superscalar Architecture

A simplified architecture is:



Multiple Execution Units

A superscalar processor may have:

- Integer ALU
- Floating-point unit
- Load/store unit
- Branch unit
- Multiply/divide unit

For example:

```
I1: ADD R1,R2,R3
I2: MUL R4,R5,R6
I3: LOAD R7,0(R8)
```

Out-of-Order Execution

In out-of-order execution, instructions do not necessarily execute in the same order in which they appear in the program.

Consider:

```
I1: LOAD R1, [A]
I2: ADD R2,R1,R3
I3: MUL R4,R5,R6
```

Instead of waiting:

$I1 \rightarrow I2 \rightarrow I3$

the processor can execute:

```
I1 —————> I2
I3 —————>
```

Thus, I3 can execute while I2 waits.

Instruction Scheduling

The scheduler identifies instructions that are ready to execute.

For example:

Instruction	Dependency	Status
I1	None	Ready
I2	I1	Waiting
I3	None	Ready
I4	I3	Waiting

The processor can execute:

Cycle 1: I1 + I3

Cycle 2: I4

Cycle 3: I2

Resource Allocation

Execution resources must be allocated efficiently.

Example:

Unit	Function
ALU1	Integer operations
ALU2	Integer operations
FPU	Floating point
LSU	Load/store
Branch Unit	Branch instructions

If two independent integer instructions are ready:

```
I1 → ALU1  
I2 → ALU2
```

Throughput Improvement

Suppose a processor has:

4 execution units

and sufficient independent instructions.

The theoretical issue rate can approach:

4 instructions/cycle

instead of:

1 instruction/cycle

But actual throughput is lower when limited by:

- Data dependencies
- Branches
- Cache misses
- Execution-unit availability
- Instruction fetch/decode bandwidth
- Memory latency

Therefore:

Actual IPC $\leq$ Maximum issue width

where IPC means Instructions Per Cycle.

Benefits of Out-of-Order Execution

1. Better Resource Utilization

Idle execution units can be used by independent instructions.

2. Higher Instruction Throughput

Multiple independent instructions can execute simultaneously.

3. Reduced Effect of Latency

The processor can perform other useful work while waiting for a slow instruction.

4. Improved Performance

Instruction-level parallelism is exploited more effectively.

Challenges

Out-of-order execution requires additional hardware and complexity, including:

- Instruction scheduler
- Reservation stations
- Register renaming
- Reorder buffer
- Dependency tracking
- Speculative execution

Register renaming is especially important because it reduces false dependencies such as WAR and WAW hazards.

Conclusion

Superscalar processors improve performance by executing multiple instructions simultaneously using multiple execution units. Out-of-order execution further increases utilization by allowing independent instructions to execute before earlier stalled instructions. However, the performance gain is limited by dependencies, branches, memory latency, and available instruction-level parallelism.